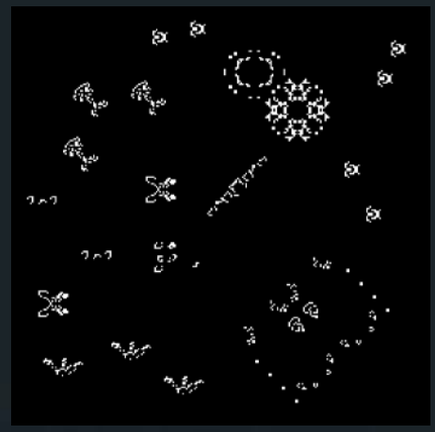
PART 3 - MODULES AND FUNCTIONS



- CONWAYS GAME OF LIFE
- CODING THE GAME
- PRACTICAL 4



zooooom....weeee...!!!

David Topping: david.topping@manchester.ac.uk

'Vanilla' Python

Data types
Basic operations

Loops
Conditionals

Pandas and data discovery

Geospatial analysis

Statistical Analysis

Cluster Analysis

1 2 3 4 5 6 7 8 9

The Numpy module

Conway's Game Of Life

Differential Equations

Modules and Functions

Creating a notebook assessment
select topic from weeks 4 - 9

10 11 12

COURSE LAYOUT >>>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

RECAP

- Functions help us repeat a set of operations many times
- They are specified using ‘def’ and ‘return’ statements

```
In [ ]: # Lets define a function called 'test'. This function will simply write a message to the screen.
def test():
    print("All work and no play makes Jack a dull boy")
    return

# Call the function
test()
```

- Modules contain functions we can access using the ‘.’ operator

```
In [ ]: import numpy as np
import pandas as pd
import sklearn #the name of the module is different from the official title
import matplotlib
```

- The Numpy module is the standard way to deal with numerical operations in Python

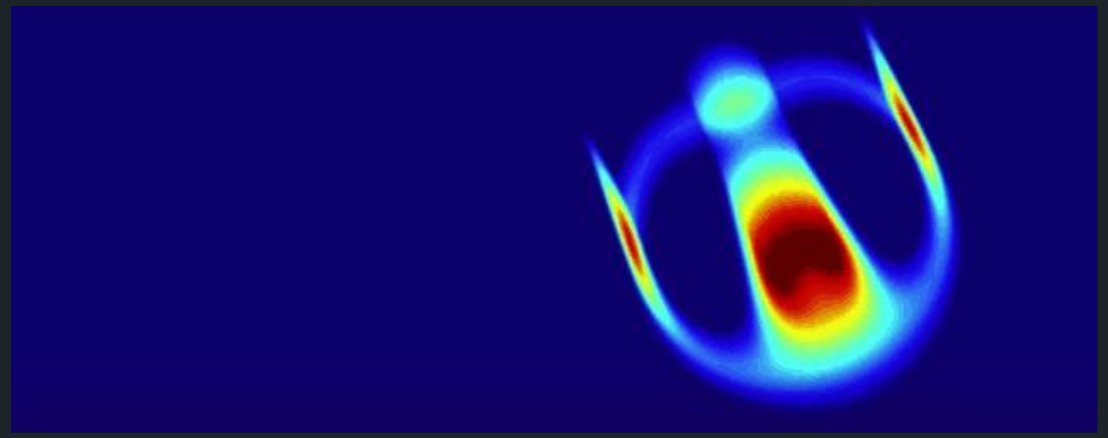
```
In [2]: import numpy as np

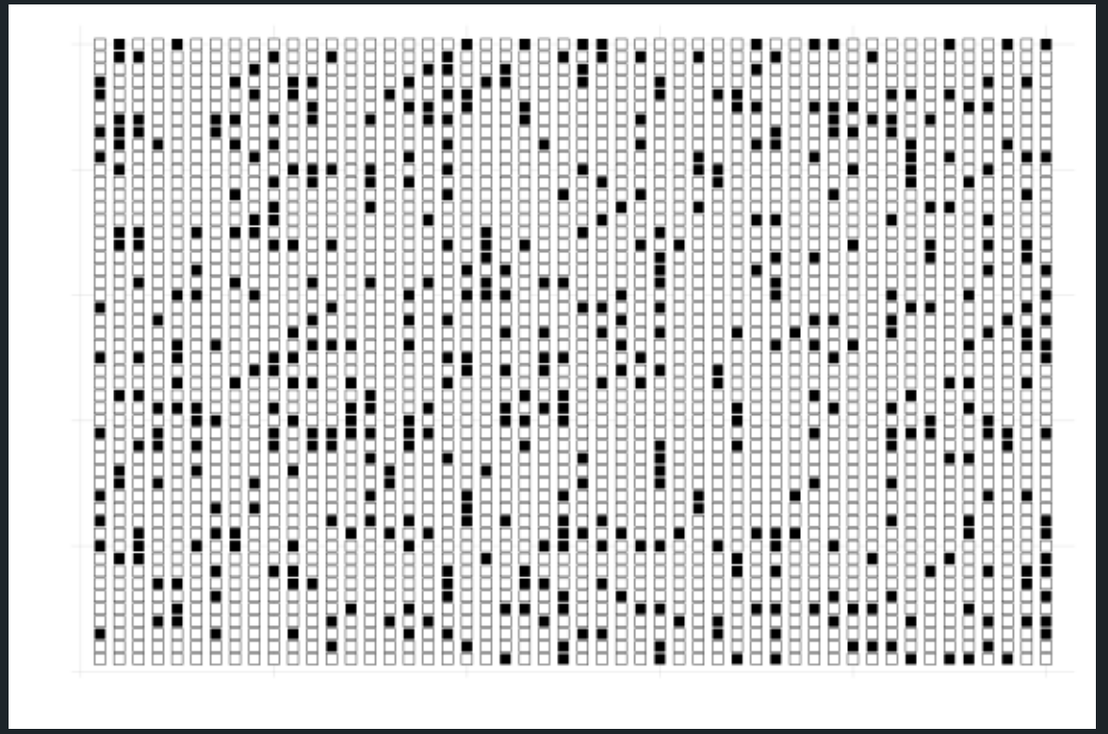
array1 = np.array([10,20,30,40,50], dtype=int)
array2 = np.array([0,1,2,3,4], dtype=int)
array3 = np.multiply(array1,array2)

print("array3 = ", array3)
```

```
array3 =  [  0  20  60 120 200]
```

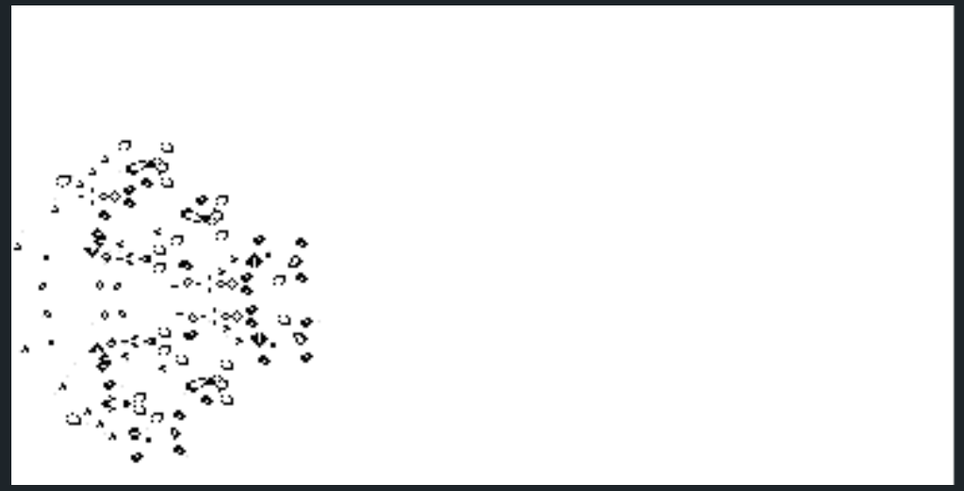
WHY DO WE RECOGNISE CERTAIN PATTERNS?





CONWAYS GAME OF LIFE

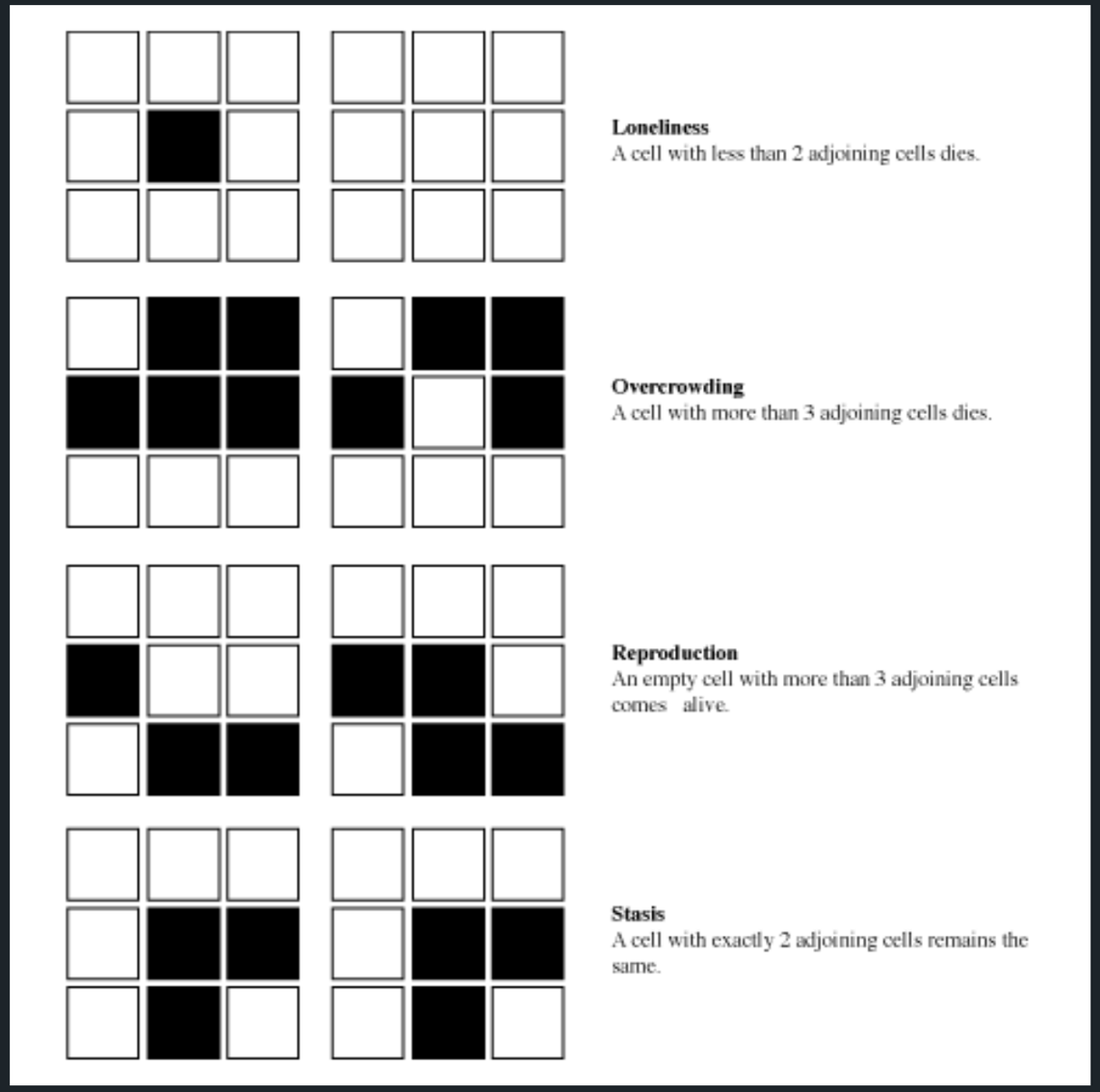
John Horton Conway (26 December 1937 - 11 April 2020) was an English mathematician active in the theory of finite groups, number theory, combinatorial game theory and coding theory. He has also made contributions to many branches of recreational mathematics, most notably the invention of the cellular automaton called the Game of Life.







CONWAYS GAME OF LIFE

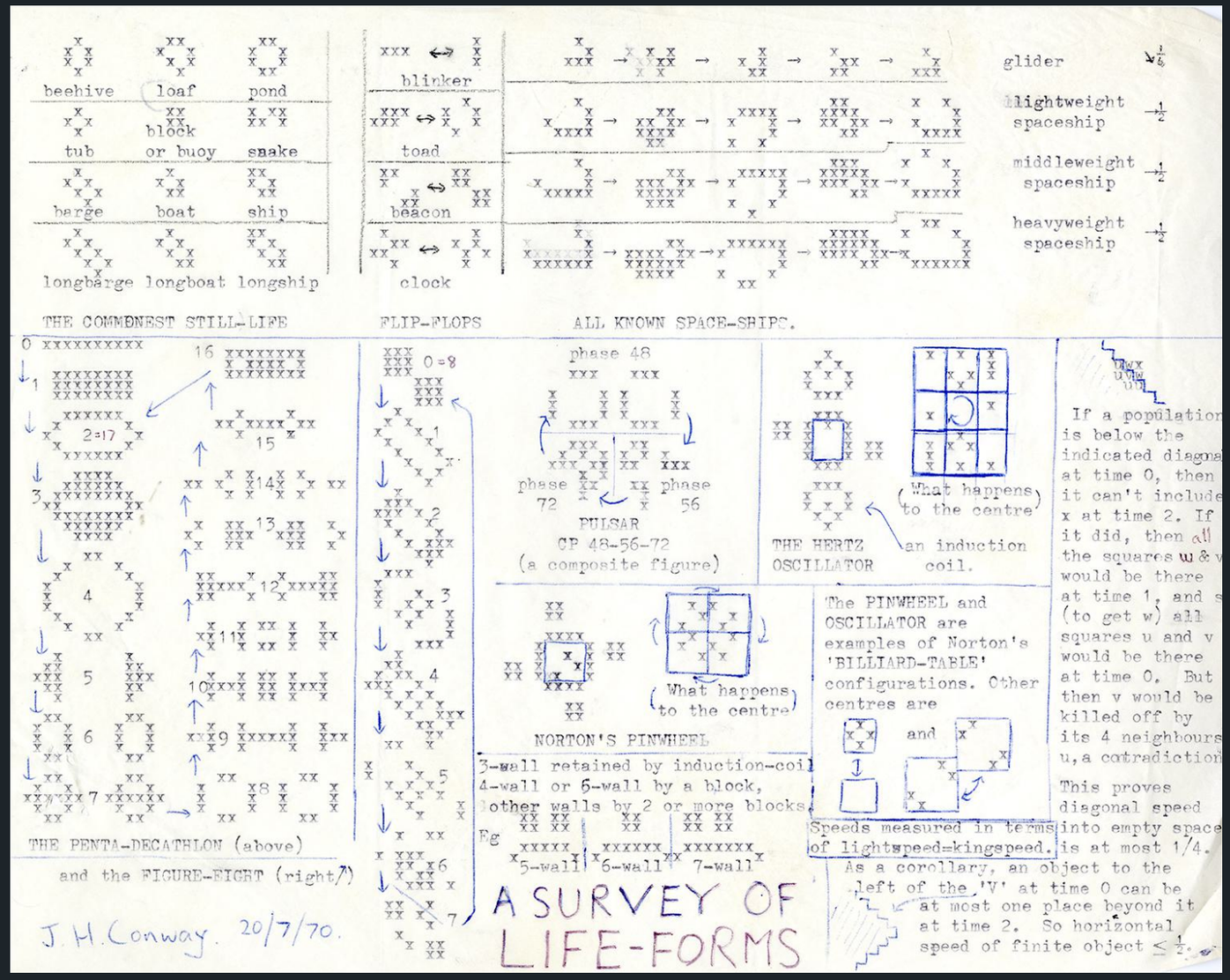


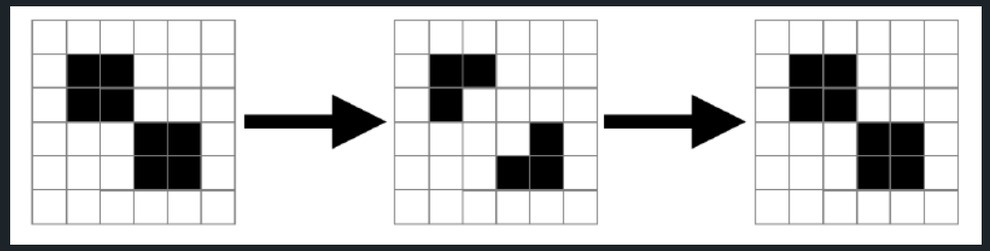
Every cell interacts with its neighbours, whether they are horizontally, vertically of diagonally adjacent. There are 4 laws that define these interactions:

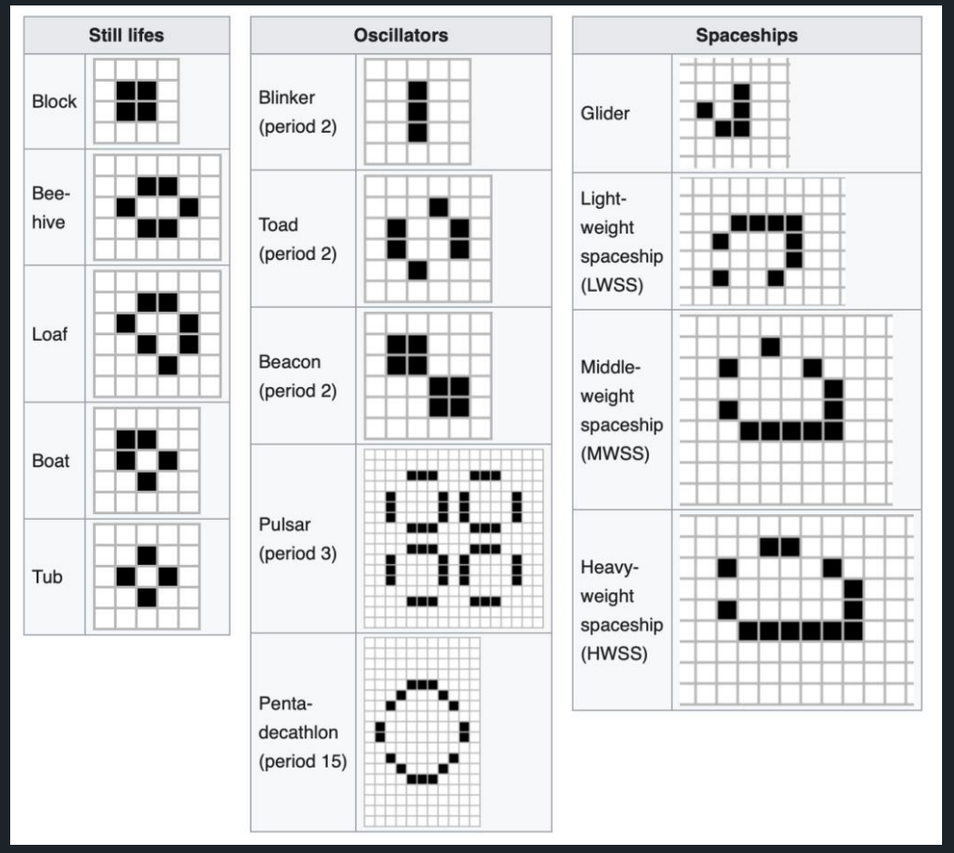
- **Any live cell with fewer than two live neighbours dies, as if by underpopulation.**
- **Any live cell with two or three live neighbours lives on to the next generation.**
- **Any live cell with more than three live neighbours dies, as if by overpopulation.**
- **Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.**

first, define a space and shape of species...

CONWAYS GAME OF LIFE



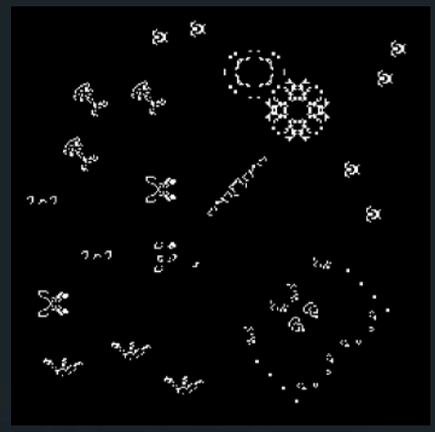




PART 3 - MODULES AND FUNCTIONS



- CONWAYS GAME OF LIFE
- CODING THE GAME
- PRACTICAL 4



zooooom....weeee...!!!

David Topping: david.topping@manchester.ac.uk

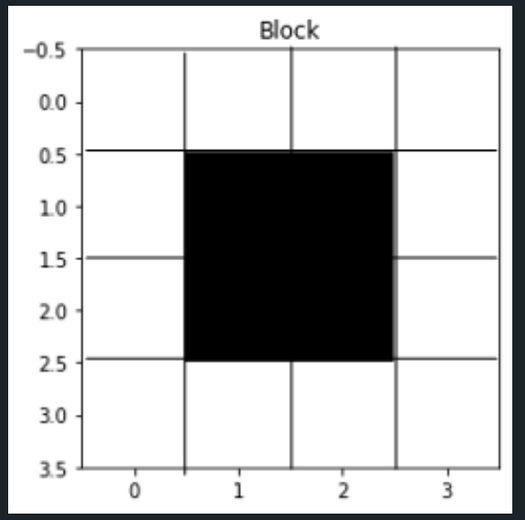
1) DEFINE A 2D ARRAY AS YOUR SPACE AND THEN POPULATE EACH VALUE WITH A 1 OR 0. E.G. A 'BLOCK', 'TUB' AND 'BOAT'

In [2]:
```
import numpy as np #import the numerical python library, numpy.
import matplotlib.pyplot as plt #as per the above, much easier to write over and over again
from matplotlib import animation, rc

# Lets first create our 'Block'. Dont forget, we can call our arrays and matrices anything we want.
Block = np.zeros((4,4),dtype=int)

# Now lets add some black cells by position some values of 1. For the Block pattern, this is done as follows:
Block[1,1]=1
Block[1,2]=1
Block[2,1]=1
Block[2,2]=1

#Now lets plot this to recreate the patterns given in figure x.
plt.imshow(Block, cmap='binary') #The cmap, or colour map, gives us a black and white board.
plt.title('Block')
plt.show()
```




```
# We have already imported both Numpy and Matplotlib so no need to import those again.
# Initialise our matrices
Tub = np.zeros((5,5),dtype=int)
Boat = np.zeros((5,5),dtype=int)

#-------'INSERT CODE HERE'-------
# Now add '1's to the currently empty 2D array Tub

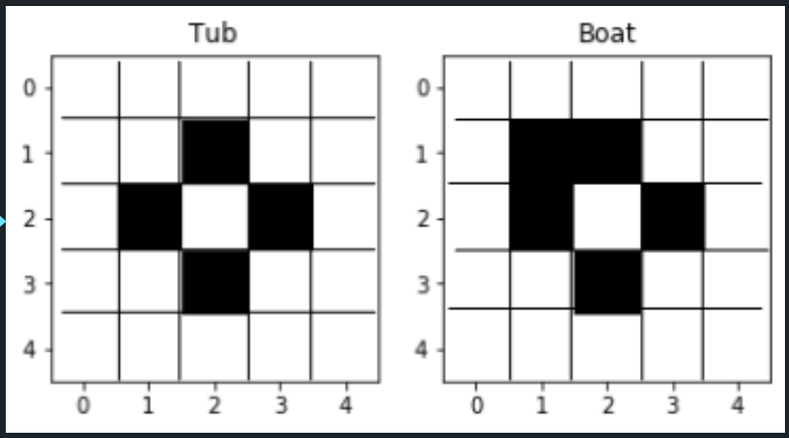
#--------------------------------

plt.subplot(1, 2, 1).imshow(Tub, cmap='binary') #The cmap, or colour map, gives us a black and white board.
plt.title('Tub')
#plt.show()

#-------'INSERT CODE HERE'-------
# Now add '1's to the currently empty 2D array Boat

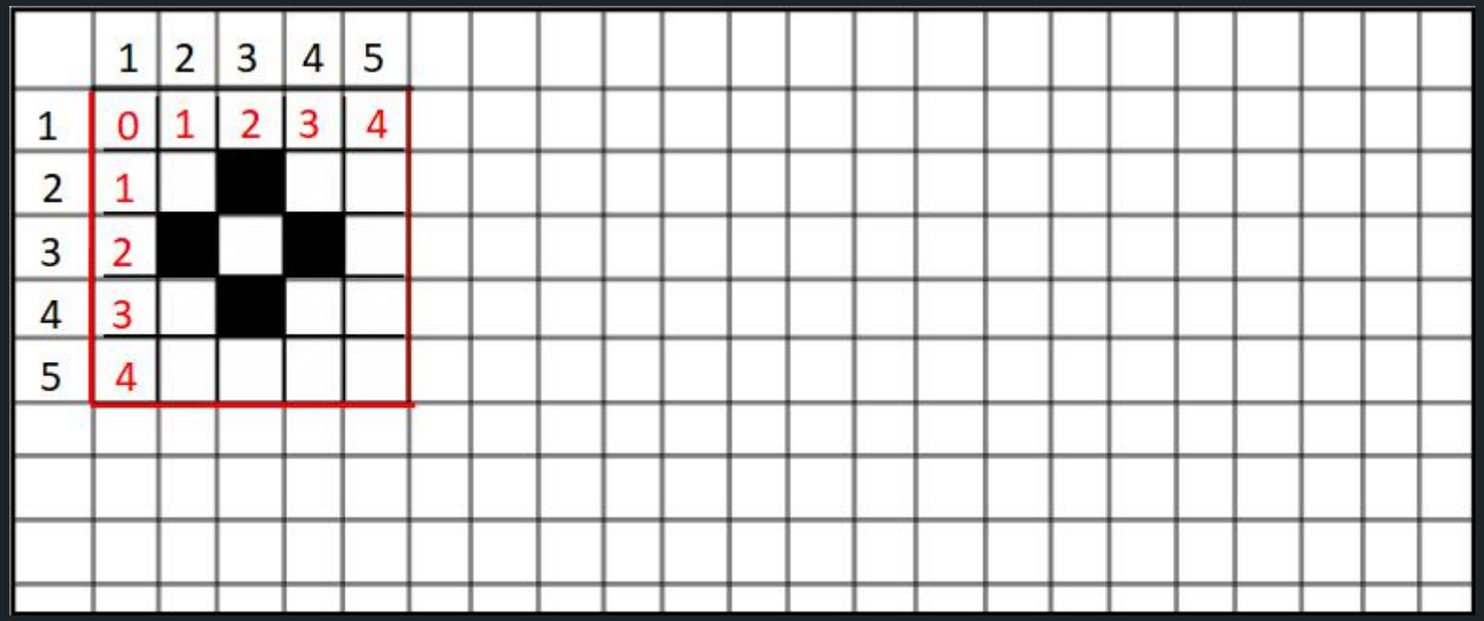
#--------------------------------

plt.subplot(1, 2, 2).imshow(Boat, cmap='binary') #The cmap, or colour map, gives us a black and white board.
plt.title('Boat')
plt.show()
```



2) POPULATE A 2D UNIVERSE WITH OUR SPECIES

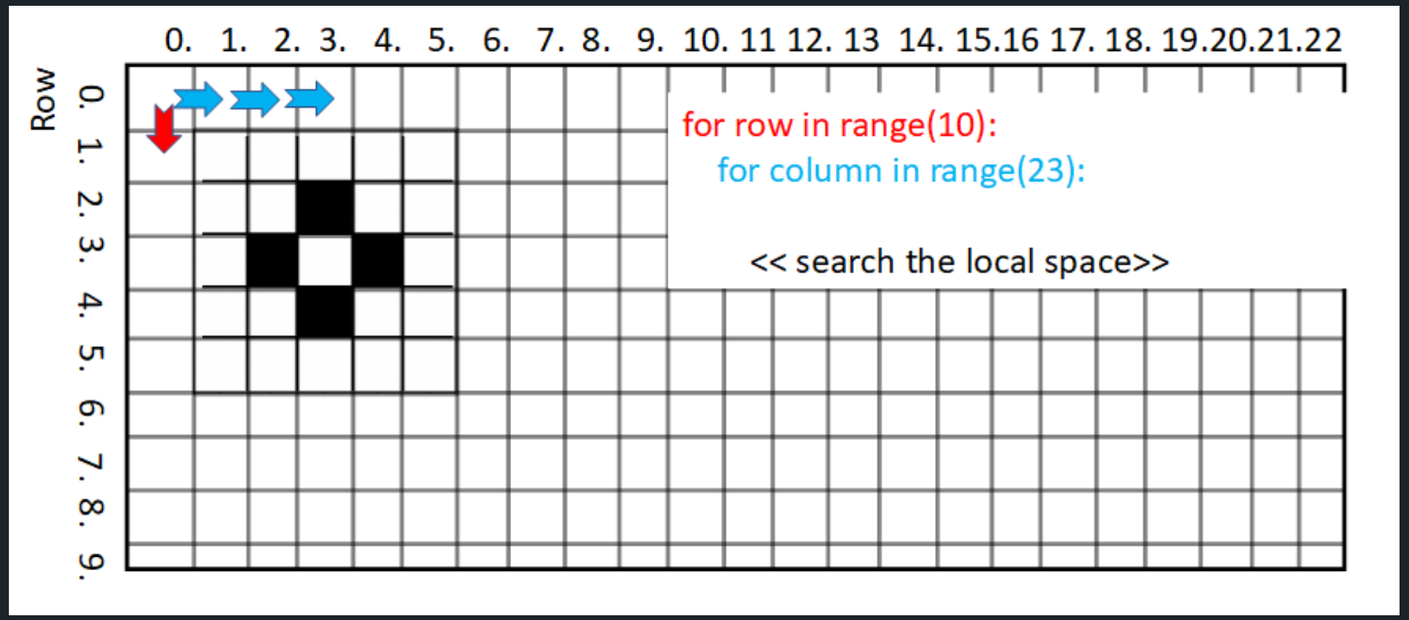
e.g. a 10 x 23 Universe



What Python commands do we need to use to initialize the above?

```
Universe = np.zeros((10,23),dtype=int)
Universe[1:5,1:5] = Tub[0:4,0:4]
```

3) SEARCH THE 2D UNIVERSE TO FIND HOW MANY LIVE OR DEAD [1 OR 0'S] NEIGHBOUR'S [CELLS] EACH CELL HAS.



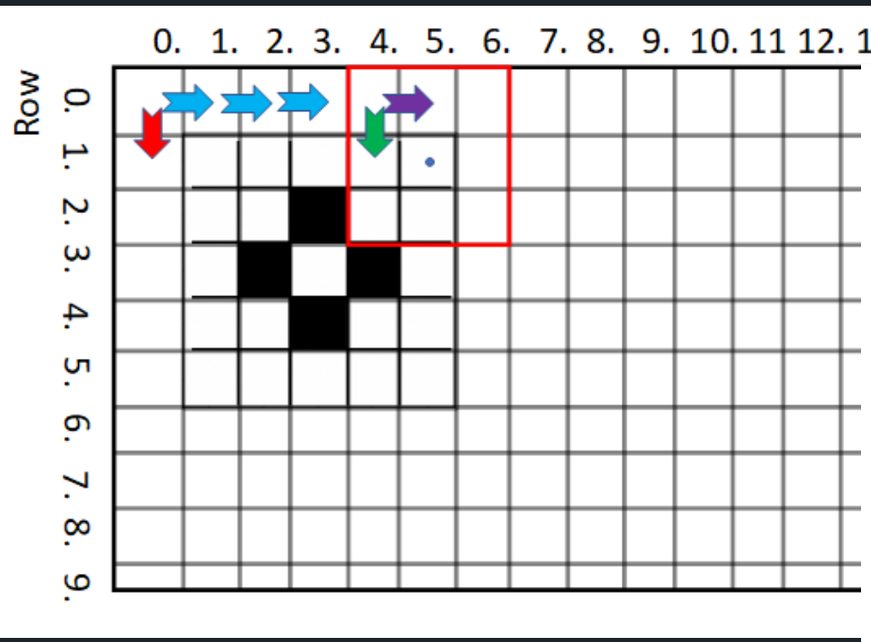
row

0	1	2	3	4	5	6

column

0	1	2	3	4	5	6

4) SEARCH THE 2D UNIVERSE TO FIND HOW MANY LIVE OR DEAD [1 OR 0'S] NEIGHBOUR'S [CELLS] EACH CELL HAS.

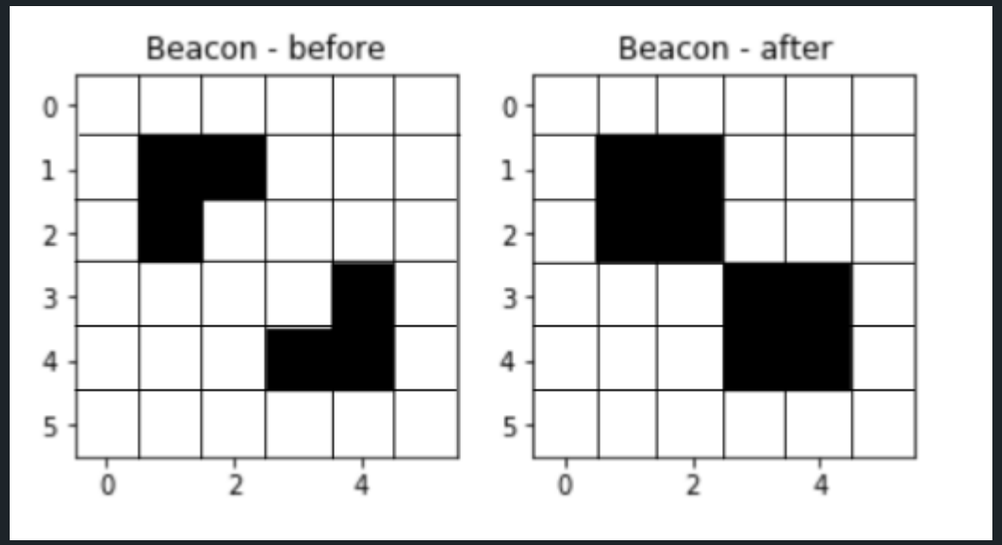



```
for row in range(10):
    for column in range(23):

        << search the local space>>
        for row2 in range(row-1,row+2):
            for col2 in range(column-1,column+2):

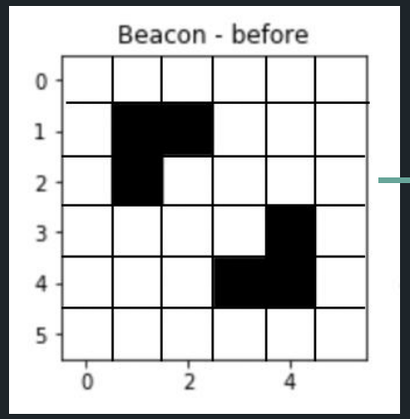
                # Set an integer variable that represents the number of alive cells
                count = 0
                if (row2<0) or (row2>=10) or (col<0) or (col2>=23):
                        # Do nothing
                    pass
                elif row2 == row and col2 == col:
                        # Do nothing, its the cell we already have!
                    pass
                # If we are not off the board or in the same cell as our starting point...
                # We can check if this new space has a value of 1. It it does, lets count it
                else:
                    if Universe[row2,col2]>0:
                            count=count+1
                # finished search the local space
```

5) PERFORM ONE PASS ACROSS THE ENTIRE 2D SPACE



- Any live cell with fewer than two live neighbours dies, as if by underpopulation.
- Any live cell with two or three live neighbours lives on to the next generation.
- Any live cell with more than three live neighbours dies, as if by overpopulation.
- Any dead cell with exactly three live neighbours becomes a live cell, as if by reproduction.

LETS USE A FUNCTION THAT WE CAN CALL OVER AND OVER.....

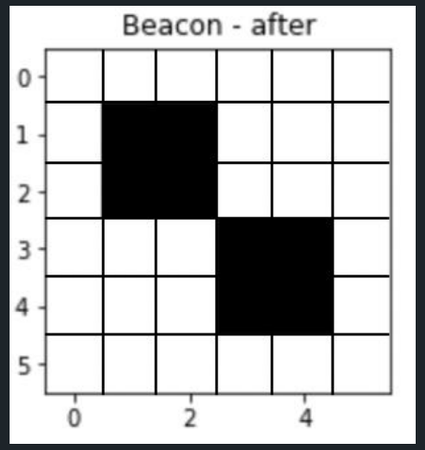



```
def search_each_cell(total_rows,total_cols,Universe_old):

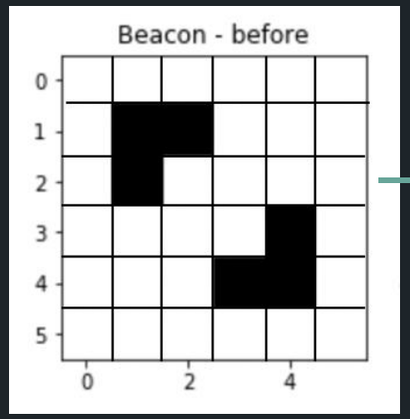
    << search and modify space>>

    return Universe_new
```

```
Universe_new = search_each_cell(total_rows,total_cols,Universe_old)
```



ITERATE OVER A FUNCTION THAT SEARCHES AND CHANGES OUR UNIVERSE.....

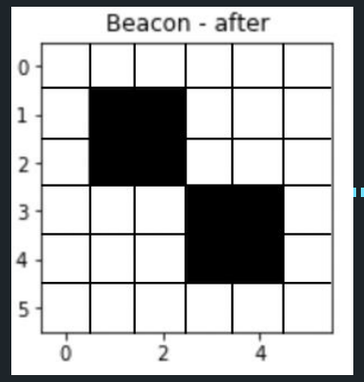


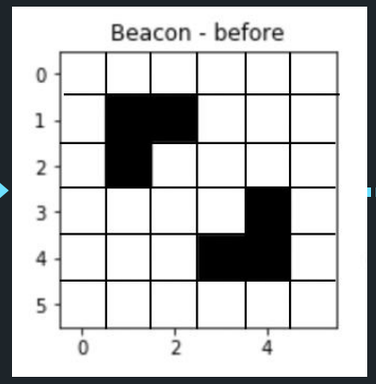

```
def search_each_cell(total_rows,total_cols,Universe_old):

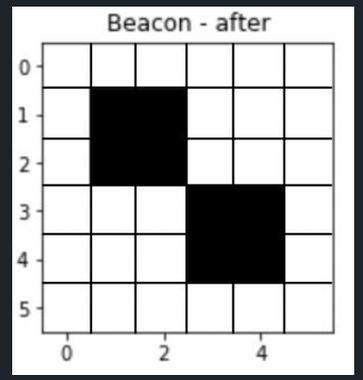
    << search and modify space>>

    return Universe_new
```

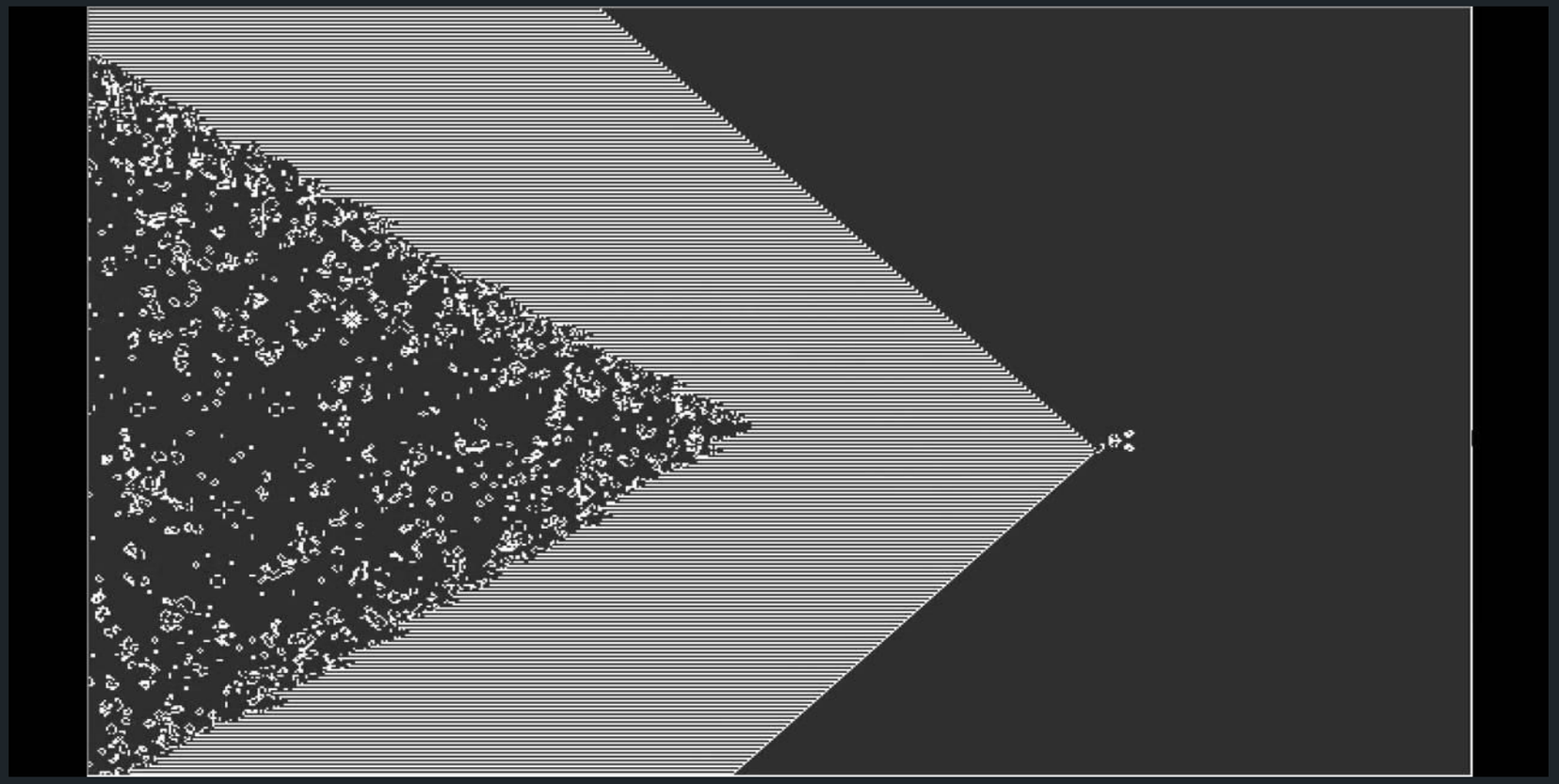
```
for step in range(20):
    Universe_new = search_each_cell(total_rows,total_cols,Universe_old)
    Universe_old = Universe_new
```



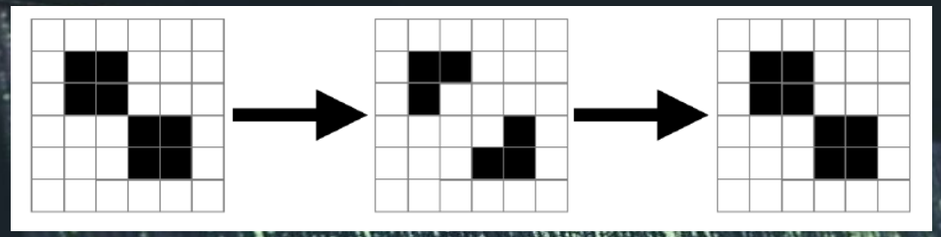


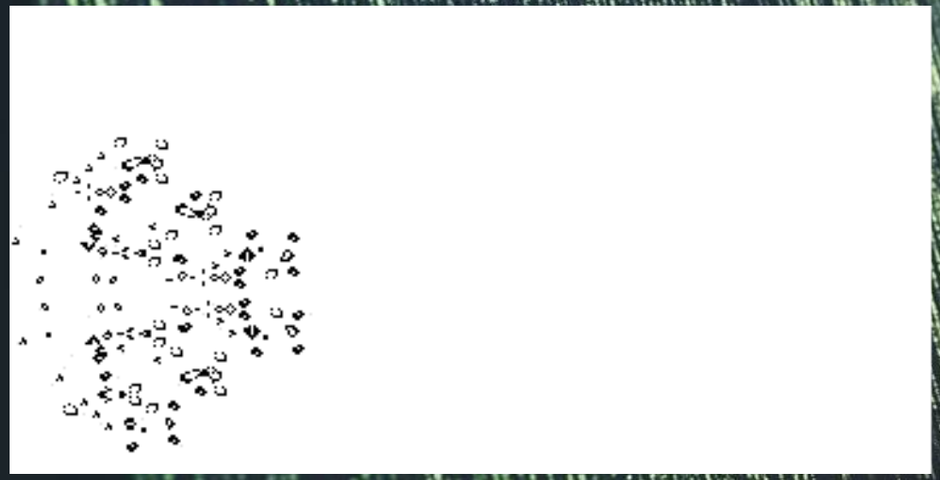


ENDLESS CREATIVE OPPORTUNITIES...



LECTURE 4



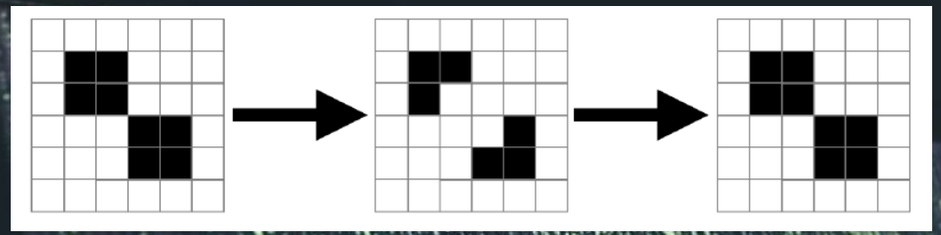


SUMMARY

CONWAYS GAME OF LIFE

- Is based on 4 rules of life in a 2D universe
- Requires us to search and count in specific locations
- Requires us to modify existing values in arrays
- Combines the above using condition statements and FOR loops

PRACTICAL 4





GOALS

CODE

- Create different shapes through 2D Numpy array modifications
- Create a function that searches a local neighbourhood for values of '1' and '0'
- Populating Conway's Universe with multiple species

GENERAL EXPERIENCE

- Learn by working on a specific problem - often the best way!
- Start to think about how you check for correct behaviour in working code